COSC 330 – Final Project: Systems Programming in Practice

Due Date: Monday (11/05/2026)
Submission: GitHub Classroom link + written report (PDF)

Objective

Demonstrate mastery of libraries, systemd, socket programming (EC2-to-EC2), and cloud storage (EC2-to-S3) in a Linux environment.

Part 1 – Interactive Shell with Custom Static & Dynamic Libraries

Task

Write a **C shell program** that presents a menu to the user. The menu should allow the user to perform **at least 3 custom operations** (e.g., add, subtract, divide, string reversal, matrix transpose, Fibonacci, etc.).
These operations must be implemented in **your own reusable library** – not just in the main file.

Library Requirements

Create **two versions** of your library:

- **Static library** (.a)
- **Dynamic (shared) library** (.so)

Your shell program must **use the shared library** at runtime.

Deliverables

- mylib.c, mylib.h
- Makefile that builds both static and shared versions
- shell.c – menu-driven program
- Written explanation of:
  - What is a **static library**? (advantages/disadvantages)
  - What is a **dynamic (shared) library**?

- What is a **shared library** (often synonymous with dynamic, but clarify .so vs .dll)
- Provide **real-world examples** for each (e.g., libc.a vs libc.so, libssl.so).

Example menu:

1. Add two numbers
2. Subtract two numbers
3. Divide two numbers
4. Exit

Part 2 – Understanding system

Task

Write a **short report + practical demo** showing how systemd works on an EC2 instance (or local Linux VM).

Requirements

- Explain: **What is systemd?** (init system, service manager, PID 1)
- Create a **custom systemd service** that runs your Part 1 shell program (or a simplified version) as a background service.
- Show commands to:
- Start/stop/enable/disable the service
- View logs using journalctl
- Explain the structure of a .service unit file.

Deliverables

- myshell.service file
- Screenshots or logs showing the service running on EC2
- 1–2 paragraph explanation of **systemd's role in modern Linux**

Part 3 – EC2 to EC2 Communication (File Transfer)

Task

Write a **client-server program** in **C or Python** running on two different EC2 instances (same VPC, same security group).

- **Server** listens for an incoming file.
- **Client** connects and sends a file (e.g., `file1.txt`) to the server.
- The server saves the file with a new name (e.g., `received_file1.txt`).

Requirements

- Use **TCP sockets** (not `scp` or `rsync`).
- Handle files up to 10 MB.
- Include error handling (connection refused, file not found, etc.).
- Explain your solution: port choice, addressing (private IPs), data framing, and any checksums.

Deliverables

- `server.c` (or `.py`)
- `client.c` (or `.py`)
- A short write-up explaining how the two EC2 instances communicated (security group rules, private IPs, etc.).

Part 4 – EC2 to S3 Communication

Task

Write a program (Python or C with AWS SDK) that **runs on an EC2 instance** and:

1. Creates a file (e.g., `output.txt`) with some content (e.g., current timestamp + system info).

2. Uploads that file to an **S3 bucket** (your own AWS account – use free tier).
3. Lists the contents of the S3 bucket.
4. Downloads the file back to EC2 (to a different name, e.g., `downloaded_output.txt`).

Requirements

- Use **AWS CLI** or **boto3 (Python)** or **libaws-c** (for C).
- **Do not hardcode credentials** – use IAM roles for EC2.
- Explain IAM role permissions (e.g., `s3:PutObject`, `s3:ListBucket`, `s3:GetObject`).

Deliverables

- `s3_transfer.py` or `s3_transfer.c`
- Screenshot of S3 bucket showing the uploaded file
- Explanation of how EC2 obtains temporary credentials (instance metadata service or IAM role)

Part 5 – Written Summary & Demonstration

Task

Write a **final report (2–3 pages)** that:

- Summarizes each part and its relevance to systems programming.
- Explains how libraries, systemd, sockets, and S3 fit into real-world cloud-native applications.
- Includes a link to a **short video demo** (max 5 minutes) showing:
    - Part 1: menu program running
    - Part 2: systemd service active
    - Part 3: file transfer between two EC2 terminals

- Part 4: file upload/download to S3

Grading Rubric (100 points total)

Part	Topic	Points
1	Shell + static/dynamic libs	25
2	systemd service & explanation	15
3	EC2 ↔ EC2 file transfer	25
4	EC2 ↔ S3	20
5	Report + demo video	15

Allowed Tools & Environment

- AWS EC2 (t2.micro, Ubuntu 22.04 or Amazon Linux 2)
- Languages: C (gcc) or Python (3.8+)
- Required packages: `gcc`, `make`, `libssl-dev` (optional), `awscli`, `boto3`

Submission

- All source code in a **GitHub repository** (private, add instructor as collaborator)
- `README.md` with compilation/run instructions
- PDF report (including explanations for each part)
- Link to demo video (unlisted YouTube or similar)